**MVP 3 is the owner-facing frontend: a simple dashboard for understanding building issues, approving technician visits, and following repairs.**

It connects to MVP 2's backend. It does not talk directly to the UNO Q, Nemotron, or Photon.

The main owner experience is:

**See the issue → understand the recommendation → approve or decline → follow progress → review the outcome.**

**1. Technology and scope**

| Item | Choice |
|---|---|
| Framework | **React + Vite** |
| Language | **JavaScript** |
| Styling | Simple CSS using the existing mockup as the reference |
| Data source | Backend HTTP API |
| Refresh | Every two seconds |
| Navigation | Overview · Activity · Sensor Data |
| User | Building owner only |

Keep this as one application with three views. Complex routing, multiple user roles, and a technician portal are unnecessary.

The frontend should work with labeled sample responses while the backend and hardware are being developed.

**2. Current building configuration**

The interface must reflect the latest hardware:

| Building | Sensors | Equipment represented |
|---|---|---|
| **Office** | One Grove light sensor | Three white LEDs and one servo representing a fan |

| | | |
|---|---|---|
| **Data Center** | One flame/IR sensor | Three green LEDs and one servo representing a fan |

Both buildings share **one Arduino UNO Q**.

Do not show:

- Two connected Arduino nodes.
- Humidity or water readings.
- Doors or knock detection.
- Measured fan speed.
- Automatic servo-health detection.

Temperature should remain hidden until its hardware and backend integration are confirmed.

**3. Overall layout**

Use the same navigation on every screen:

**BuildingLink Overview Activity Sensor Data Owner**

The owner area can show a name or simple avatar. It does not need a role selector.

Use:

- White cards on a light background.
- Dark, readable text.
- Teal for primary actions.
- Amber for attention.
- Green for normal or resolved states.
- Icons and text together, rather than relying on colour alone.

Avoid filling the screen with technical details. The main screen should explain the situation in ordinary language, with supporting information available when expanded.

**4. Overview tab**

This is the default screen.

Its main purpose is to answer:

1. Are my buildings okay?
2. Is there anything I need to approve?
3. What is happening with service?

**A. Page summary**

Display a short heading and a sentence based on actual state:

| Situation | Example |
|---|---|
| Nothing needs attention | “No requests need your attention.” |
| One approval pending | “One request needs your approval.” |
| Technician assigned | “A technician has accepted your service request.” |
| Readings unavailable | “Building readings are temporarily unavailable.” |

Don’t display “Everything is normal” when the controller has stopped reporting.

**B. Two building cards**

Each card shows:

- Building name and icon.
- Main status.
- One short explanation.
- Last reading update.
- A **View details** action.

| Status | Meaning |
|---|---|
| **Normal** | Fresh readings and no unresolved issue requiring attention |
| **Needs attention** | An active issue or pending owner decision |
| **Service in progress** | Approved work is being coordinated or performed |
| **Status unavailable** | No fresh readings and no reliable current assessment |

If an incident is already open when readings stop, preserve the incident and add **“Latest readings unavailable.”** Don’t hide the issue behind a disconnected badge.

**C. Approval request card**

This is the most prominent element whenever approval is pending.

Show:

| Field | What the owner sees |
|---|---|
| Building | Office or Data Center |
| Issue | A short, understandable title |
| Explanation | What was observed |
| Recommendation | What the technician would inspect |
| Uncertainty | What remains unconfirmed |
| Decision | Approve technician visit / Decline |

Example:

> **Office lighting needs inspection**
> Light levels remain below the usual range while the lights are expected to be on. An obstruction or lighting problem may be responsible.
> **Recommended action:** Request a maintenance inspection.

Buttons:

- **Approve technician visit**
- **Decline**

Underneath:

**“We will contact a technician after you approve.”**

Do not invent a price, appointment time, or guaranteed diagnosis.

**D. “Why am I seeing this?”**

Use an expandable section containing:

- The observation that triggered the incident.
- The checks already performed.
- A relevant previous repair, if one was found.
- Why that repair is relevant.
- What still needs inspection.

Show concise findings and tool results, not an internal reasoning transcript.

For example:

> “The lights are expected to be on, but recent readings remain below the calibrated baseline. A previous case involved a loose lighting connection on comparable equipment. The current cause is unconfirmed.”

A lighting repair should not appear as evidence explaining an unrelated IR alert.

**E. Service progress**

After approval, replace the decision controls with progress:

**Approved → Contacting technician → Visit accepted → Repair reported → Verified and closed**

Only mark a stage complete after the backend records it.

Use clear exception states:

- Awaiting technician response.
- Technician declined.
- Contact attempt failed.
- Repair awaiting verification.

An answered or completed call does not automatically mean the technician accepted the job.

**5. Approval interaction**

Approval is a real application action, not a local visual toggle.

Recommended interaction:

1. Owner clicks **Approve technician visit**.
2. A small confirmation dialog repeats the building and proposed work.
3. Owner confirms.
4. Disable the controls while submitting.
5. Send the request ID, proposal version, and decision to the backend.
6. Wait for the backend response.
7. Refresh state immediately.
8. Continue normal polling.

Do not show “Approved” before the backend accepts the decision.

If approval was already submitted through Photon, the next refresh should show that result. Both routes act on the same request.

**Decline behavior**

After decline:

- Show **“Visit declined.”**
- Explain that no technician visit was authorized.
- Keep the unresolved issue visible.
- Remove active approval buttons for that request.

Declining service is not the same as fixing the problem.

**6. Activity tab**

Activity is a chronological history of important events.

Each entry should contain:

| Field | Example |
|---|---|
| Building | Office |
| Event | “You approved a technician visit.” |
| Time | Local date/time or relative time |
| Related issue | “Office lighting inspection” |
| Status | Awaiting technician response |
| Details action | Open incident history |

Useful event types:

- Issue detected.
- Investigation started or completed.
- Owner clarification requested.
- Approval request sent.
- Owner approved or declined.
- Technician contact attempted.
- Technician accepted.
- Repair findings received.
- Verification completed.
- Issue resolved.
- Controller connection lost or restored.

Ordinary sensor readings should not create individual Activity entries.

**Filters**

Keep filtering small:

- **Building:** All / Office / Data Center.
- **Issue status:** All / Open / Resolved.

The status filter applies to the related issue, rather than pretending every individual event has an open/resolved lifecycle.

**Incident details**

Clicking an event opens a drawer or expanded panel showing:

- Original evidence.
- Investigation summary.
- Owner decision.
- Technician-contact history.
- Findings.
- Relevant previous repair.
- Verification result.

A drawer avoids needing a separate page for every incident.

**7. Sensor Data tab**

This is the detailed measurement view.

Its main table should contain:

| Building | Sensor | Latest reading | Assessment | Last received |
|---|---|---|---|---|
| Office | Light level | Raw ADC value | Within expected range / Needs inspection / Not calibrated | Relative timestamp |
| Data Center | Flame/IR | Triggered / Not triggered | IR alert / No current alert | Relative timestamp |

These are the current confirmed sensor categories.

**Office light details**

When selected, show:

- Raw value.
- Configured ADC range, if available.
- Recent history.
- Calibrated baseline, if available.
- Whether office lights are expected to be on.

- Data source and freshness.

Use an explanation such as:

> “Shows the amount of light reaching the sensor. Shadows and surrounding room lighting can affect the reading.”

Don’t display lux or a percentage unless there is an explicitly defined conversion or calibration.

**Data-center IR details**

Show:

- Current normalized trigger state.
- Last detected activation.
- Recent activation history.
- Data freshness.
- Whether the input is live or simulated.

Use:

> “An infrared signal was detected. This does not independently confirm a fire.”

A simple event list or binary history is more appropriate than a continuous temperature-style chart.

**History display**

For the light sensor, a simple line chart is useful after the core workflow works. Label the vertical axis **Raw reading**.

If chart implementation delays integration, start with a timestamp/value table. Use actual history from the backend rather than a decorative generated graph.

**Equipment commands**

A separate read-only section can show:

| Building | Requested equipment state |
|---|---|
| Office | Three lights requested on; fan model requested running |
| Data Center | Three lights requested on; fan model requested stopped |

Label it **Commanded states**.

If the controller has acknowledged a command, show that separately. Never convert acknowledgment into “physical movement verified.”

**8. Backend API consumption**

The frontend should use a small API module rather than calling `fetch` throughout every component.

| Backend route | Frontend purpose |
|---|---|
| `GET /api/state` | Overview, connection state, pending requests, service progress |
| `GET /api/activity` | Activity timeline |
| `GET /api/buildings/:id/readings` | Sensor history |
| `GET /api/incidents/:id` | Expanded incident details |
| `POST /api/service-requests/:id/decision` | Owner approval or decline |

Optional:

- An investigation retry action if the backend supports it.
- Authorized demo hardware commands, kept outside the normal owner flow.

These are the proposed MVP 2 routes. Confirm the exact response shapes with the backend developer before building around them.

The browser must not call Photon or Nemotron directly. Their credentials stay in the backend.

**9. Polling and synchronization**

Use these rules:

- Fetch current state on page load.
- Refresh approximately every two seconds.
- Avoid overlapping requests if one is still running.
- Refresh immediately after a successful owner decision.
- Cancel or ignore outdated requests when changing views.
- Preserve open drawers and filters during refresh.
- Don’t wipe the screen while waiting for the next response.

There are two different freshness questions:

1. **Can the browser reach the backend?**
2. **Is the backend receiving fresh UNO Q data?**

Display them separately.

A working backend connection does not prove the sensor data is current.

If a request fails, retain the last known values but visibly mark them stale. Do not present them as live.

**10. Loading, empty, and error states**

Design these explicitly:

| Situation | Owner-facing treatment |
|---|---|
| Initial load | “Loading your buildings…” |
| Backend unreachable | “Unable to refresh. Showing the last received information.” |
| UNO Q unavailable | “Building readings are unavailable.” |
| No sensor data yet | “Waiting for the first reading.” |
| Missing calibration | “Baseline has not been set.” |
| No open incidents | “No issues currently need your attention.” |
| No activity | “Building updates will appear here.” |
| No matching repair case | “No relevant previous repair was found.” |
| Investigation failed | “We couldn’t complete the assessment. Please retry.” |
| Approval submission failed | “Your decision wasn’t confirmed. Refresh before trying again.” |
| Proposal changed | “This request has changed. Review the latest recommendation.” |
| Technician did not respond | “Awaiting technician response.” |

Don’t expose stack traces, credentials, or raw provider errors to the owner.

**11. Suggested component structure**

Keep the implementation small:

| Component | Responsibility |
|---|---|
| `App` | Navigation and top-level state |
| `AppHeader` | Product name, tabs, owner indicator |
| `OverviewPage` | Building summaries and current requests |
| `BuildingCard` | One building's status |
| `ApprovalCard` | Explanation and decision actions |
| `ApprovalDialog` | Confirmation before submission |
| `ServiceProgress` | Recorded workflow stages |
| `EvidencePanel` | Expandable supporting information |
| `ActivityPage` | Filters and event timeline |
| `IncidentDetails` | Full history in a drawer |
| `SensorDataPage` | Sensor table and selected reading |
| `SensorHistory` | Recent reading table or chart |
| `ConnectionBanner` | Backend/device freshness warnings |

Supporting modules:

- `api.js` for backend requests.
- `formatters.js` for readings and timestamps.
- A small polling hook.
- Central CSS.
- Labeled mock fixtures.

No complex global state library is necessary for this scope. Keep filters, selected tabs, dialogs, and loading states local where practical.

### 12. Example frontend/backend contract

Agree on the summary structure with MVP 2. At minimum, the frontend needs:

| Record | Required information |
|---|---|
| **Device** | ID, connection status, last received time |
| **Building** | ID, name, status, short explanation |
| **Reading** | Sensor ID, value, unit/type, timestamp, source |
| **Approval request** | ID, version, building, proposed action, evidence summary, status |
| **Work order** | ID, status, technician display name, latest update |
| **Repair reference** | Source building, confirmed finding, outcome, relevance |
| **Activity event** | ID, building, type, description, timestamp, incident ID |

Render unknown enum values safely as an understandable fallback rather than crashing.

Use the server's record IDs for actions. Don't identify a request by its position in a displayed list.

### 13. Basic access and safe rendering

The frontend operates as the configured owner.

- Use the backend's owner-session mechanism.
- Do not submit an arbitrary `owner: true` flag and treat it as authorization.
- Prevent repeated submissions while a decision is processing.
- Let the backend enforce idempotency and approval rules.
- Render agent and technician text as text, rather than injecting raw HTML.
- Avoid saving integration credentials in browser storage.

The small confirmation dialog helps prevent accidental approval, but the backend remains authoritative.

### 14. Demo controls

These are useful for development and judging but should remain visually separate from owner decisions.

Possible controls:

- Office LEDs on/off.
- Data-center LEDs on/off.
- Office fan-model movement on/off.
- Data-center fan-model movement on/off.
- Clearly labeled simulated incident input.

Use an explicit development/demo setting to expose them. Hiding controls in the UI is not a substitute for backend authorization.

Never show simulated inputs as live hardware measurements.

**15. Build sequence**

Build in this order:

1. Create the application shell and three tabs.
2. Implement building cards with labeled fixture data.
3. Build approval cards and the decision dialog.
4. Connect `GET /api/state`.
5. Connect approval/decline submission.
6. Add service progress and error states.
7. Build Activity and incident details.
8. Build Sensor Data using actual backend readings.
9. Add history visualization if time permits.
10. Test Photon-originated approval updates appearing through polling.
11. Add clearly separated demo controls only if required.

The frontend can progress before hardware integration by using sample responses matching the agreed API contract.

**16. Acceptance tests**

| Test | Expected behavior |
|---|---|
| Fresh data, no incidents | Both building cards show normal conditions. |
| Pending request | Correct building and proposed work are visible. |
| Approve clicked twice | Controls prevent repeated submission; backend receives one intended decision. |
| Approval received through Photon | Dashboard updates without requiring manual reload. |

| | |
|---|---|
| Owner declines | Request changes to declined; unresolved issue stays visible. |
| Technician call completed without acceptance | UI continues waiting for an explicit response. |
| Technician accepts | Service progress advances. |
| Repair reported complete | UI shows verification pending. |
| Backend verifies resolution | Issue becomes resolved and appears in Activity. |
| UNO Q stops reporting | Both buildings' readings become unavailable/stale. |
| Browser loses backend connection | Last-known information remains visibly stale. |
| Simulated reading received | Demo/source label is visible. |
| Only office has a light sensor | No data-center light measurement is invented. |
| No fan feedback exists | No RPM or measured working-status metric appears. |
| Narrow screen or keyboard navigation | Approval controls remain usable and clearly labeled. |

**17. Handoff deliverables**

The frontend developer should provide:

- Working React application.
- The three tabs.
- API module and agreed response assumptions.
- Approval/decline interaction.
- Service progress and Activity history.
- Sensor readings and freshness handling.
- Loading, empty, and error states.
- Labeled fixtures for independent development.
- Responsive layout.
- A short run guide and backend-URL configuration.
- A list of remaining integration gaps.

**MVP 3 is complete when the owner can understand an incident, approve or decline the proposed visit, follow actual service updates, and inspect sensor evidence—without needing to understand Arduino wiring, agent tools, or API details.**